Skill 09: Software Application Development

Module 3: Duty & Charges Engine

1. Contents

Module 3 consists of the following files:

1. `Module 3.pdf` (this document)
2. `assets/fx-rates.csv` — daily exchange rates against the colón
3. `assets/duty-rates.csv` — import duty and selective consumption tax rates by HS-6
4. `assets/trade-agreements.csv` — the trade agreements in force and the countries in each
5. `assets/trade-preferences.csv` — preferential duty rates by agreement and HS-6
6. `assets/antidumping-measures.csv` — anti-dumping measures in force
7. `NortransOne/` — the skeleton solution: `Contracts` (fixed) and `Services` (your work)

The same skeleton is already the first commit of your `Module-3` repository.

2. Introduction

You have **2 hours 30 minutes** to implement a .NET 8 class library for Nortrans.

This is the module Rodrigo Alfaro has been waiting three years for.

Every import declaration Nortrans files is priced by an Excel workbook that Esteban Vargas wrote in 2011. It has 40 sheets, 11 macros, and one cell where a hard-coded exchange rate from 2018 still lives. When a rate changes, somebody edits the workbook. When somebody edits the workbook, the next declaration is wrong until somebody notices. Last quarter a misapplied preferential rate cost a client ₡2.4 million and cost Nortrans a client.

Your job is to replace that workbook with a library: given a commercial invoice and its lines, produce the customs value and every charge due, to the céntimo, reproducibly.

#	Component	Reads	Produces
1	Chargeable weight calculator	cargo pieces with dimensions and weights	air chargeable weight, ocean revenue tons
2	Customs valuation service	an invoice, its lines, freight and insurance	the CIF customs value of the declaration and of each line
3	Duty assessment engine	a valued declaration	duty, taxes, totals and findings per line and for the declaration

The three components form a pipeline: the chargeable weight feeds the freight charge, the customs valuation feeds the duty assessment. Implement them in the order given.

3. Submission

Push to the main branch of the repository `Module-3`. Your submission contains:

- the complete solution, with your implementations in `Services/`;
- a `README.md` with notes for the marking team, including every assumption you made where you judged the specification to leave a gap.

Only the `Services` folder is marked. Do not rename or move it. Do not modify the `Contracts` project. Implement exactly one class per interface; if there are several, only the first in alphabetical order is used.

4. Guidelines

These apply to every part of this module.

- **Money is always an integer number of minor units.** Colón amounts are in **céntimos** (1/100 of a colón). Foreign currency amounts are in the minor unit of that currency (cents for USD and EUR). Never expose a `double` or a `decimal` across the contract.
- **Rounding is away from zero**, to the whole minor unit, and it is applied at every step that the specification names as a step — not only at the end. `0.5` becomes `1`, `-0.5` becomes `-1`.
- **Rates are in basis points.** One basis point is 0.01 %. A rate of `1400` is 14 %. A charge is `value × rateBasisPoints / 10000`, rounded away from zero.
- Weights are integer **grams**. Dimensions are integer **millimetres**. The library is metric throughout.
- All dates and times are **UTC instants**.
- **No method throws.** Malformed, missing or contradictory input produces the documented result, with the relevant finding code attached. An unhandled exception during marking loses the aspect.
- A result's total must always equal the exact sum of its parts. Where rounding makes that impossible, the specification says where the difference goes. Follow it: the marking team checks the totals against the parts.
- The lookup tables are CSV files in `assets/`. Copy them into your project and set them to be copied to the output directory.

5. Part 1 — Chargeable weight calculator

Interface: `IChargeableWeightCalculator`.

A consignment is a list of `CargoPiece` records:

```
CargoPiece(int Quantity, int LengthMm, int WidthMm, int HeightMm, int
GrossWeightGrams)
```

`GrossWeightGrams` is the weight of **one** piece. `Quantity` is how many identical pieces there are.

5.1 Air chargeable weight

```
ChargeableWeight AirChargeableWeight(IReadOnlyList<CargoPiece>? pieces)
```

1. The volumetric weight of one piece, in grams, is `Length × Width × Height / 6000` with the dimensions in millimetres, rounded away from zero to whole grams. *(This is the IATA divisor of 6000 cm³ per kilogram, expressed in millimetres.)*
2. The volumetric weight of the consignment is the sum over all pieces, each multiplied by its quantity.
3. The gross weight of the consignment is the sum of `GrossWeightGrams × Quantity`.
4. The chargeable weight is the greater of the two, **rounded up to the next whole 500 grams**. A chargeable weight that is already a multiple of 500 is not rounded up further.
5. The result reports gross weight, volumetric weight and chargeable weight, all in grams.

A null or empty list returns a result with all three values zero and the finding `NO_PIECES_SUPPLIED`. A piece with a negative or zero dimension, or a negative weight, is skipped and adds the finding `INVALID_PIECE`; the remaining pieces are still calculated. A piece with `Quantity` less than 1 is treated the same way.

5.2 Ocean revenue tons

```
OceanMeasure OceanRevenueTons(IReadOnlyList<CargoPiece>? pieces)
```

Less-than-container-load ocean freight is charged on whichever is greater, the cubic metres or the metric tonnes — the “revenue ton”, also called weight-or-measure.

1. The volume of the consignment in **cubic millimetres** is the sum of `Length × Width × Height × Quantity`.
2. Express the volume in thousandths of a cubic metre: divide by 1 000 000, rounded away from zero.
3. Express the gross weight in thousandths of a metric tonne: divide the total grams by 1 000, rounded away from zero.
4. The revenue ton, in thousandths, is the greater of the two, **with a minimum of 1 000** (one revenue ton is the minimum charged).
5. The result reports volume, weight and revenue tons, all in thousandths of their unit.

The same handling of null, empty and invalid pieces as in 5.1 applies.

6. Part 2 — Customs valuation service

Interface: `ICustomsValuationService`.

```
CustomsValuation Value(ValuationRequest? request)
```

The customs value is the **CIF** value: the price paid for the goods, plus the cost of getting them to the border, plus insurance — the transaction value method of the WTO Customs Valuation Agreement. Everything is converted to colones and apportioned across the declaration lines.

6.1 Currency conversion

1. Every amount in the request is expressed in `InvoiceCurrency`, in that currency's minor unit.
2. Look up the rate in `fx-rates.csv`. The file has one row per currency and date, with the column `rateCentimosPerUnit`: how many céntimos one **whole unit** of the foreign currency is worth. A rate of `52340` means one US dollar is ₡523.40.
3. Use the row whose date equals the declaration date. If there is no such row, use the row with the **most recent earlier date** for that currency — rates do not change at weekends and holidays, but the file only carries banking days.
4. If the currency is `CRC`, no conversion happens and no lookup is needed.
5. If the currency is not `CRC` and no row on or before the declaration date exists, return a valuation with every amount zero and the finding `FX_RATE_NOT_FOUND`. Do not guess a rate.
6. To convert: `centimos = minorUnits × rateCentimosPerUnit / 100`, rounded away from zero.

6.2 Insurance

If `InsuranceAmountMinor` is `null`, the declaration has no declared insurance and a notional premium is used: **1.25 % of the FOB amount**, in the invoice currency, rounded away from zero, and the finding `NOTIONAL_INSURANCE_APPLIED` is added. If `InsuranceAmountMinor` is supplied — including a supplied zero — it is used as it is.

6.3 Apportionment across lines

Freight and insurance are declared for the whole consignment and must be split across the lines. They are apportioned **by gross weight**, because that is what Nortrans is billed on.

1. For each line, its share is `total × lineGrossWeightGrams / totalGrossWeightGrams`, rounded away from zero.
2. Because of the rounding, the shares may not add up to the total. The difference — positive or negative — is added to the line with the **greatest gross weight**. If two lines tie on gross weight, it goes to the one with the **lowest line number**.
3. If the total gross weight of all lines is zero, apportion by FOB value share instead, with the same residual rule applied to the line with the greatest FOB amount, and add the finding `APPORTIONED_BY_VALUE`.
4. If the total FOB is also zero, split equally, give the residual to the lowest line number, and add the finding `APPORTIONED_EQUALLY`.

Freight and insurance are apportioned **separately**, each with its own residual, not as one combined figure.

6.4 Result

For each line: FOB in céntimos, apportioned freight, apportioned insurance, and the customs value (the sum of those three).

For the declaration: the customs value, which must equal the exact sum of the line customs values, and the list of findings.

A null request, an empty line list, or a negative FOB, freight or insurance amount returns a valuation with every amount zero and the finding `INVALID_REQUEST`.

7. Part 3 — Duty assessment engine

Interface: `IDutyAssessmentEngine`.

```
DutyAssessment Assess(DutyAssessmentRequest? request)
```

The request carries the valued declaration from part 2, the declaration date, the customs regime, and the per-line origin and certificate information.

7.1 De minimis — checked first, for the whole declaration

If the regime is `CustomsRegime.Courier` **and** the total customs value of the declaration is **3 000 000 céntimos (₡30 000) or less**, no charge is due. Return an assessment in which every charge on every line is zero, with the finding `DE_MINIMIS_APPLIED`. Nothing in 7.2 to 7.6 is computed.

De minimis never applies to any other regime, whatever the value.

7.2 Import duty (DAI)

```
dai = lineCustomsValue × daiRateBasisPoints / 10000
```

The general rate comes from `duty-rates.csv`, keyed by the **first six digits** of the line's HS code. If the HS-6 is not in the table, the duty rate is `0`, the selective consumption tax rate is `0`, and the finding `HS_CODE_NOT_IN_TARIFF` is added for that line — the line is still assessed.

Preferential origin. A lower rate applies when all of these are true:

1. the line's origin country appears in `trade-agreements.csv` as a party to an agreement in force on the declaration date;
2. `HasOriginCertificate` is `true`;
3. `CertificateIssuedUtc` is not null, is not in the future relative to the declaration date, and the declaration date is **not more than 365 days after** it;
4. a row exists in `trade-preferences.csv` for that agreement and that HS-6.

If all four hold, the preferential rate from `trade-preferences.csv` replaces the general rate and the finding `PREFERENCE_APPLIED` is added for the line. If condition 1 and 4 hold but the certificate is missing, the general rate applies and the finding is `PREFERENCE_DENIED_NO_CERTIFICATE`. If the certificate exists but fails condition 3, the general rate applies and the finding is `PREFERENCE_DENIED_CERTIFICATE_EXPIRED`.

If a country is a party to more than one agreement that covers the HS-6, use the **lowest** preferential rate available.

7.3 Law 6946 levy

`law6946 = lineCustomsValue × 100 / 10000` — that is, 1 % of the customs value.

If `duty-rates.csv` marks the HS-6 as exempt (`law6946Exempt = true`), the levy is zero and the finding `LAW6946_EXEMPT` is added for the line.

7.4 Selective consumption tax (SCT)

```
sct = (lineCustomsValue + dai) × sctRateBasisPoints / 10000
```

The rate comes from `duty-rates.csv`. Note that the base includes the import duty already calculated: this is a cascading tax.

7.5 Anti-dumping surcharge

If a row in `antidumping-measures.csv` matches the line's **HS-6 and origin country**, and the declaration date falls on or between that row's `inForceFrom` and `inForceTo` dates (both inclusive; an empty `inForceTo` means the measure is open-ended), then:

```
antiDumping = lineCustomsValue × surchargeBasisPoints / 10000
```

and the finding `ANTIDUMPING_APPLIED` is added for the line. The anti-dumping surcharge is **not** part of the base for any other charge.

7.6 Value added tax (IVA)

`vat = (lineCustomsValue + dai + law6946 + sct) × 1300 / 10000` — 13 %.

The anti-dumping surcharge is excluded from the VAT base.

7.7 Totals

- The line total is `dai + law6946 + sct + antiDumping + vat`.
- The declaration total for each charge is the exact sum of that charge across the lines.
- The declaration grand total is the exact sum of the line totals.
- No rounding is applied at declaration level: each charge was already rounded at line level.

7.8 Invalid input

A null request, or a request whose valuation is null or has no lines, returns an assessment with every amount zero and the finding `INVALID_REQUEST`.

8. Worked example

Declaration `NTX-2026-0417`, regime `General`, declaration date `2026-03-11T00:00:00Z`, invoice currency `USD`, FOB `USD 12 400.00` (`1240000` cents), freight `USD 860.00`, insurance not declared, two lines:

Line	HS-6	Origin	FOB (cents)	Gross (g)	Certificate
1	`847130`	`US`	`900000`	`180000`	yes, issued `2025-11-02`
2	`392690`	`CN`	`340000`	`420000`	no

Expected intermediate values, so that you can check your implementation:

Step	Value
FX rate for USD on 2026-03-11	`52340` céntimos per USD
Notional insurance	1.25 % of `1240000` = `15500` cents
FOB in céntimos	`1240000 × 52340 / 100` = `649016000`

Step	Value
Freight in céntimos	`86000 × 52340 / 100 = 45012400`
Insurance in céntimos	`15500 × 52340 / 100 = 8112700`
Line 1 freight share (180 000 / 600 000)	`13503720`
Line 2 freight share (420 000 / 600 000)	`31508680`
Line 1 customs value	`471060000 + 13503720 + 2433810 =` **`486997530`**

The rest is for you to derive. The full expected result for this example is held by the marking team.

Time: 2:30 hours · Skill 09 Competition Simulation · Nortrans Cargo & Customs